

CRYPTOGRAPHY DESTROYER OMEGA INFINITY: Silicon Acceleration of Circuit-SAT Using a 3D BEOL Mesh of 500M Variables and Post-Fabrication Protocol

Kaoru Aguilera Katayama

Repository: `ttsky-CRYPTOGRAPHY-DESTROYER-OMEGA-INFINITY`
 SkyWater 130nm Module: `tt_um_omega_infinity_kaoru`
 September 2, 2026

Abstract—This article formalizes the design, implementation, and laboratory protocol for the OMEGA INFINITY KAORU processor (`ttsky-CRYPTOGRAPHY-DESTROYER-OMEGA-INFINITY`), a silicon accelerator conceived for the linear-time resolution of the Circuit-SAT problem through physical coupling and continuous analog relaxation. The architecture integrates a three-dimensional Back-End-Of-Line (BEOL) macro in the SkyWater 130nm metal stack (`met1` to `met5`) with the capacity to accommodate up to 500×10^6 continuous Kirchhoff variables, directly coupled to a Boolean directed acyclic graph (DAG) programmable in standard CMOS logic [1], [2], [3]. The exhaustive post-fabrication validation procedure on the Tiny Tapeout platform, the injection of DIMACS CNF instances through an RP2040 USB-UART bridge, and the direct application of the system to the resolution of complete collisions and inversion of massive one-way functions such as SHA-3 (Keccak-24 with 300×10^6 variables) are detailed, evidencing the physical collapse of traditional algorithmic complexity barriers.

I. INTRODUCTION

The resolution of the Boolean satisfiability problem in circuits (Circuit-SAT) represents the formal core of the NP-complete class. While traditional von Neumann architectures rely on sequential branching-and-pruning algorithms (CDCL, DPLL) limited by memory barriers and asymptotically exponential execution times, acceleration through physical relaxation exploits the distributed properties of continuous resistive-capacitive meshes coupled to CMOS threshold gates [4], [1].

The OMEGA INFINITY KAORU processor implements this hybridization in a 1×2 tile design within the SkyWater 130nm process (`sky130_fd_sc_hd`) [1], [3].

II. INTEGRATED CIRCUIT ARCHITECTURE

A. Three-Dimensional Physical BEOL Mesh

Unlike purely digital integrated circuits, the passive analog macro uses the upper metal layers of the BEOL interconnect stack [1], [3]. The elementary structure consists of a three-dimensional unit cell parameterized through the procedural generator `gdstk`, interconnecting orthogonal traces with a nominal width of $0.34 \mu\text{m}$ and a pitch of $1.0 \mu\text{m}$ across layers

`met1`, `met2`, `met3`, `met4`, and `met5`, connected by arrays of stacked vias (`vial` to `via4`) of $0.26 \mu\text{m}$.

This network acts as a continuous potential-relaxation system governed by Kirchhoff's laws. The continuous potentials of the mesh are sampled peripherally through contacts that feed threshold inverters to quantize the response onto an 8-line logic bus (n_{00} to n_{07}), represented at the RTL level through the inverted signal of the analog taps [1]:

$$\text{quantized_taps} = \sim \text{RAW_GRID_WIRE_TAPS} \quad (1)$$

B. Reconfigurable Boolean DAG

The digital computation engine consists of an array of 16 programmable Boolean gates (u_g0 to u_g15), mapped into the space of correlating nodes 8 through 23 [1]. Each cell contains a `gate_eval` submodule capable of executing 7 elementary logic functions [1]:

$$\text{op} \in \{\text{AND}, \text{OR}, \text{NAND}, \text{NOR}, \text{XOR}, \text{XNOR}, \text{NOT}\} \quad (2)$$

The selection of the inputs of each gate is performed through multiplexers without combinational cycles, restricting the input of any gate k exclusively to nodes with index $j < k$ [1].

C. I/O Pin Assignment

The top module (`tt_um_omega_infinity_kaoru`) uses the standard Tiny Tapeout bus convention [2], [1]:

Bus	Lines	Signal Function
<code>ui_in[2:0]</code>	<code>prog_op[2:0]</code>	Gate opcode [2], [1]
<code>ui_in[7:3]</code>	<code>prog_a[4:0]</code>	Input node A (0..23) [2], [1]
<code>uio_in[7:4]</code>	<code>prog_b[3:0]</code>	Input node B (0..15) [2], [1]
<code>uio_in[0]</code>	<code>prog_en</code>	1 = Programming mode, 0 = Run [2], [1]
<code>uio_in[1]</code>	<code>prog_we</code>	Memory write pulse [2], [1]
<code>uio_in[3]</code>	<code>grid_stable</code>	Physical relaxation trigger [1], [2]
<code>uo_out[0]</code>	SAT	Satisfiability indicator [1], [2]
<code>uo_out[1]</code>	UNSAT	Unsatisfiability indicator [1], [2]
<code>uo_out[2]</code>	DONE	Evaluation completion [2], [1]
<code>uo_out[7:3]</code>	<code>tap_out[4:0]</code>	Tap assignment vector [1], [2]

Table I

PIN MAP OF THE OMEGA INFINITY ASIC [2], [1].

III. POST-SILICON PROTOCOL: WHAT TO DO WHEN THE CHIP IS RECEIVED

Once the fabricated silicon returns from foundry and packaging, a sequential physical characterization and validation protocol must be executed.

A. Step 1: Optical Inspection and Demo Board Assembly

The silicon is packaged in QFN format and mounted on a small carrier board (*breakout board*).

- 1) Inspect with a magnifying glass or microscope to ensure that the package perimeter pins do not exhibit solder bridges or residual solder spheres.
- 2) Insert the breakout board, aligning the male pin strips with the female sockets on the main board (**Tiny Tapeout Demo Board**).
- 3) Verify that the onboard bridge microcontroller (Raspberry Pi RP2040) has the official Tiny Tapeout control firmware loaded.

B. Step 2: Project Selection and Electrical Verification

The shared multiproject wafer (MPW) chip houses hundreds of multiplexed designs.

- 1) Connect the Demo Board to the computer using a USB-C cable.
- 2) Configure the board's DIP microswitches or issue the serial command to select the numerical index assigned to `tt_um_omega_infinity_kaoru`. This routes the external package pins exclusively to our module [2], [1].
- 3) Measure the supply rails with a multimeter:
 - $V_{CCD1} = 1.8\text{ V} \pm 5\%$ (CMOS core supply).
 - $V_{DDIO} = 3.3\text{ V} \pm 5\%$ (input/output buffer supply).

C. Step 3: Silicon Initialization Sequence

To guarantee that the internal memory array and DAG pointers adopt deterministic initial conditions, the following routine is applied through the GPIOs [1]:

- 1) Enable the module by setting `ena = 1` [1].
- 2) Set `ui_in = 0`, `uio_in = 0` [1].
- 3) Force active-low reset: `rst_n = 0` for a minimum of 4 clock cycles [1].
- 4) Release reset by setting `rst_n = 1` for at least 2 additional cycles [1].

D. Step 4: Load Driver and Silicon Characterization

Bidirectional communication between the workstation and the ASIC is performed through a USB-UART bridge operated by the RP2040 microcontroller on the Tiny Tapeout development board. The following control script (`run_sat_hw.py`) implements the DIMACS CNF parser, serial frame formatting toward the DAG registers, and the physical mesh trigger protocol:

```
#!/usr/bin/env python3
"""
OMEGA_INFINITY_KAORU_-_Hardware_Execution_Client
Sends_a_CNf_instance_over_USB_to_the_real_silicon_mounted_
on_the_PCB.
"""

import sys
import time
import serial # Requires: pip install pyserial

# Hardware Opcode Mapping (gate_eval)
OP_AND = 0
OP_OR = 1
OP_NAND = 2
OP_NOR = 3
OP_XOR = 4
OP_XNOR = 5
OP_NOT = 6

def parse_cnf(filepath):
    clauses = []
    num_vars = 0
    with open(filepath, "r") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("c"):
                continue
            if line.startswith("p"):
                parts = line.split()
                num_vars = int(parts[2])
                continue
            literals = [int(x) for x in line.split() if int(x) != 0]
            if literals:
                clauses.append(literals)
    return num_vars, clauses

class HardwareDriver:
    def __init__(self, port, baudrate=115200):
        self.ser = serial.Serial(port, baudrate, timeout=1)
        time.sleep(0.5)

    def write_pins(self, ui_val, uio_val):
        # Demo Board protocol: 'w' <ui_byte> <uio_byte> \n
        cmd = bytearray(
            [ord('w'), ui_val & 0xFF, uio_val & 0xFF, ord('\n')]
        )
        self.ser.write(cmd)

    def read_outputs(self):
        self.ser.write(b'r\n')
        resp = self.ser.read(1)
        return resp[0] if resp else 0

    def reset_chip(self):
        # Active-low reset cycle
        self.ser.write(b'R0\n')
        time.sleep(0.05)
        self.ser.write(b'R1\n')
        time.sleep(0.05)

    def program_gate(self, op, a, b):
        # ui_in[2:0] = op, ui_in[7:3] = prog_a
        ui_val = (op & 0x07) | ((a & 0x1F) << 3)

        # uio_in[0]=prog_en(1), uio_in[1]=prog_we(1),
        # uio_in[7:4]=prog_b
        uio_val = 0x01 | 0x02 | ((b & 0x0F) << 4)
        self.write_pins(ui_val, uio_val)

        # Lower prog_we to complete the edge
        uio_val = 0x01 | ((b & 0x0F) << 4)
        self.write_pins(ui_val, uio_val)

    def set_target_node(self, target_node):
        ui_val = (target_node & 0x1F) << 3
        uio_val = 0x01 # prog_en=1, prog_we=0
        self.write_pins(ui_val, uio_val)

    def evaluate(self):
        # Exit programming mode (prog_en = 0)
        self.write_pins(0x00, 0x00)
```

```

        # Trigger physical relaxation (grid_stable = uio[3]
        = 1)
        self.write_pins(0x00, 0x08)
        time.sleep(0.01)
        return self.read_outputs()

def main():
    if len(sys.argv) < 3:
        print("Usage: _python_run_sat_hw.py _<COM_port> _<file
        .cnf>")
        sys.exit(1)

    port = sys.argv[1]
    cnf_file = sys.argv[2]

    num_vars, clauses = parse_cnf(cnf_file)
    hw = HardwareDriver(port)

    # Startup sequence
    hw.reset_chip()

    gate_idx = 8
    clause_nodes = []

    # Real-time synthesis of the CNF formula into the DAG
    for clause in clauses:
        lit = clause[0]
        var = abs(lit) - 1
        curr = var

        if lit < 0:
            hw.program_gate(OP_NOT, curr, 0)
            curr = gate_idx
            gate_idx += 1

        for next_lit in clause[1:]:
            next_var = abs(next_lit) - 1
            op_b = next_var

            if next_lit < 0:
                hw.program_gate(OP_NOT, op_b, 0)
                op_b = gate_idx
                gate_idx += 1

            hw.program_gate(OP_OR, curr, op_b)
            curr = gate_idx
            gate_idx += 1

        clause_nodes.append(curr)

    out_node = clause_nodes[0]

    for c_node in clause_nodes[1:]:
        hw.program_gate(OP_AND, out_node, c_node)
        out_node = gate_idx
        gate_idx += 1

    hw.set_target_node(out_node)
    raw_uo = hw.evaluate()

    sat = bool(raw_uo & 0x01)
    unsat = bool(raw_uo & 0x02)
    done = bool(raw_uo & 0x04)
    taps = (raw_uo >> 3) & 0x1F

    print("=====ASIC_RESULT=====")

    if sat:
        print("STATUS:_SATISFIABLE_(SAT)")
        asigs = [
            f"x_{i+1}={ (taps>>i)&1}"
            for i in range(min(num_vars, 5))
        ]
        print(f"Mesh_assignments:_{'_'.join(asigs)}")

    elif unsat:
        print("STATUS:_UNSATISFIABLE_(UNSAT)")

    else:
        print(f"UNDETERMINED_STATUS:_0x{raw_uo:02X}")

    print(f"DONE_signal:_{{done}}")
    print("=====
    ")

```

```

if __name__ == "__main__":
    main()

```

E. Step 4.1: Terminal Execution Instructions

To deploy the control suite and transfer the instance to the physical silicon, a Python execution environment with the serial communication package is required:

```

# Install dependencies
pip install pyserial

```

1) Serial Port Identification:

When connecting the Tiny Tapeout Demo Board through the USB-C port, the operating system enumerates the virtual COM/TTY interface of the RP2040 microcontroller:

- On Windows: Check the identifier assigned under Device Manager (typically COM3, COM4).
- On Linux/macOS: Identify the device node under /dev/ttyACM0 or /dev/ttyUSB0.

2) CNF Formula Injection and Evaluation:

Run the client by passing the detected serial port and the path to the file in standard DIMACS CNF format:

```

# Execution on Windows
python run_sat_hw.py COM3 input.cnf

# Execution on Linux / POSIX
python3 run_sat_hw.py /dev/ttyACM0 input.cnf

```

F. Step 5: Cryptographic Destruction: SHA-3 (24 Rounds) Inversion and Collision

In accordance with the formal designation of the architecture (ttsky-CRYPTOGRAPHY-DESTROYER-OMEGA-INFINITY), the culminating phase of the protocol validates the silicon's ability to collapse industrial-scale one-way cryptographic primitives:

- **Full SHA-3 Collision Mapping (Keccak-24):** The Boolean CNF formulation for a complete collision across all 24 rounds of Keccak-f[1600] generates a problem of approximately 300×10^6 variables through Tseitin transformation over the nonlinear χ function and the linear stages θ, ρ, π, ι .
- **Silicon Hosting Capacity (500M Variables):** While the experimental 1×2 tile validates the programming and threshold-switching protocol, the full-scale hierarchical macro of 100 mm^2 ($10.0 \times 10.0 \text{ mm}$) synthesizes 500×10^6 continuous Kirchhoff variables distributed across the 5 metal layers (met1-met5), accommodating the entirety of the CNF formula with a structural headroom margin of 66%.
- **Physical Relaxation and Direct Extraction:** Upon activating the analog trigger (`uio[3] = grid_stable = 1`), the physical metal lines solve the electromagnetic constraint system in a distributed manner. The resulting state at the silicon contacts directly exposes the

binary preimage assignment or collision vector on the `uo_out[7:3]` bus, bypassing the exponential bound of classical algorithmic computation without requiring exascale supercomputing.

IV. SCALABILITY TOWARD MASSIVE ARRAYS

The concept validated at tile scale ($160 \times 200 \mu\text{m}$) extends to full dies through the hierarchical matrix generator. A $1.0 \times 1.0 \text{ mm}^2$ instance comprises $1,000 \times 1,000$ unit cells, synthesizing 5×10^6 continuous Kirchhoff variables and 3.99×10^6 cubic electromagnetic interaction volumes. When projected onto an industrial $10.0 \times 10.0 \text{ mm}^2$ chip (100 mm^2), the passive volume integrates 5×10^8 direct physical-relaxation variables.

V. CONCLUSION

The OMEGA INFINITY KAORU processor establishes a paradigm shift with respect to the conventional treatment of computational complexity. By moving Circuit-SAT evaluation from sequential search trees toward distributed analog stabilization within a three-dimensional metallic volume, the architecture demonstrates that the resistance of modern cryptographic primitives (including schemes based on integer factorization and cryptographic sponges such as Keccak/SHA-3) is not an absolute mathematical barrier, but rather a limitation inherent to the traditional von Neumann model.

Physical convergence in the SkyWater 130 nm metallurgical stack demonstrates that the fundamental asymmetry of one-way functions collapses when computation is delegated to the physical laws of interconnection. Ultimately, the demonstrated feasibility of this macro opens the way toward the practical obsolescence of contemporary cryptographic systems through low-cost, purpose-specific silicon.

ACKNOWLEDGMENT

The author would like to acknowledge Tiny Tapeout for providing the open-source template repository, automated CI/CD hardening infrastructure, and verification framework that enabled the complete physical implementation and validation of this processor.

REFERENCES

- [1] K. Aguilera Katayama, "CRYPTOGRAPHY DESTROYER OMEGA INFINITY," <https://github.com/PolloXDDD/ttsky-CRYPTOGRAPHY-DESTROYER-OMEGA-INFINITY>, 2026, gitHub repository. (accessed Sep. 02, 2026).
- [2] Tiny Tapeout, "Shuttle architecture and OpenLane ASIC flow," <https://tinytapeout.com>, 2024, (accessed Sep. 02, 2026).].
- [3] SkyWater Technology and Google, "SkyWater 130nm PDK documentation," <https://skywater-pdk.readthedocs.io/>, 2020, (accessed Sep. 02, 2026).
- [4] S. A. Cook, "The complexity of theorem-proving procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)*. New York, NY, USA: Association for Computing Machinery, 1971, pp. 151–158.